BOB-169 â€” root cause PROVEN (acceptance (a) satisfied)

Captured 2026-08-21. The filed item asked whoever took it to â€œdetermine by invocation which generator emits fragments and under what conditionsâ€¦. That is now answered from source plus a falsifiable prediction that held, so acceptance (a) can be closed rather than repeated.

The generator

`scripts/generate_markdown_exports.sh:57`

```
pandoc -f markdown -t html5 -o "$html" "$md" --metadata title="$(basename
```

No `--standalone` / `-s`. Without it pandoc emits a BODY FRAGMENT â€” no `<!DOCTYPE>`, no `<html>`, no `<head>`, and therefore no `<meta charset>`.

A tell that the flag was intended and lost: `--metadata title=` is passed on that same line, and the title can only ever render inside a standalone documentâ€™s `<head><title>`. As written the script computes a title it is structurally incapable of emitting.

The two branches disagree, and the FALLBACK is the correct one

The `else` branch (python-markdown, used only when pandoc is ABSENT) writes:

```
<!DOCTYPE html><html><head><meta charset="utf-8"><title>{title}</title></h
```

So the LESS-preferred path is well-formed and the PREFERRED path is defective. The consequence is inverted from the usual: a host WITHOUT pandoc produces correct exports, and a properly-provisioned host produces broken ones. That is why this survived â€” the defect is invisible on exactly the machines least likely to be treated as degraded.

The PDF mechanism â€” proven, no longer hypothesis

`generate_markdown_exports.sh:76`

```
weasyprint "$html" "$pdf"
```

The PDF is rendered FROM the fragment HTML. Given no encoding declaration, weasyprint applies its fallback, and the UTF-8 bytes are decoded as latin-1 and BAKED INTO the PDF text layer as glyphs. An earlier

revision of BOB-169 recorded this as an untraced hypothesis; it is now read directly from the source chain.

The falsifiable prediction that confirms it

DOCX is generated on a THIRD path â€” `pandoc -f markdown -t docx` at line 85, direct from markdown, never touching the HTML. If the chain above is correct, the DOCX must be CLEAN while its .html and .pdf siblings are not. Measured on docs/BOBA_DATABASE.*:

```
.docx  ->  0 mojibake sequences,  7 clean Â§ characters   (word/document.x
.pdf   ->  'Ã,Â§ 5', 'JackettÃ¢â‚¬â„¢s', 'Ã¢â€ '                    (pdfto
.html  ->  no DOCTYPE, no charset, 30 lines of non-ASCII
```

The DOCX is the control. It shares the same source markdown and the same pandoc binary, and differs only in NOT passing through the charset-less HTML â€” and it is clean. That isolates the defect to the HTML step and rules out a corrupt source or a broken pandoc.

The fix

Add `-s` (or `--standalone`) to line 57. pandocâ€™s default html5 standalone template emits `<meta charset="utf-8" />` â€” verified empirically, not assumed: when docs_chain regenerated docs/features/Status.html with a standalone pipeline, the diff was 183 insertions / 0 deletions and the inserted preamble contained exactly that meta tag.

Then regenerate. Note the blast radius honestly: 286 .html files plus their PDFs change in one commit, and every PDF must be re-rendered AFTER the HTML is fixed â€” regenerating PDFs first would re-bake the same mojibake.

What is still NOT measured

How many of the ~300 PDFs carry baked mojibake. One was probed. The 286-file census covered .html only. Count it; do not extrapolate.

Nuance discovered after the fact: the generator is mtime-guarded

`generate_markdown_exports.sh` only (re)generates when

```
[[ ! -f "$html" || "$md" -nt "$pdf" ]]
```

Measured consequence: a full `workable-items-export.sh` run AFTER docs_chain had rewritten docs/features/Status.html as a standalone document did NOT revert it â€” `docs_chain verify --all` still reported both contexts in-sync. The export skipped the file because its HTML is now newer than its source.

Two implications for the fix:

1. Adding `-s` to line 57 will NOT heal the 286 existing fragments. They will be regenerated only when their .md is next touched â€” so the corpus would heal silently, unevenly, and over an unbounded period, with no point at which anyone can say it is done. The fix therefore needs a FORCED regeneration pass, not just the flag.
2. It also explains why this defect is so old and so quiet: files are rewritten only on source change, so a fragment minted once persists indefinitely, and the corpus accumulated 286 of them one document at a time.

Order matters in that forced pass: HTML must be regenerated BEFORE the PDFs, because weasyprint reads the HTML. Regenerating PDFs first re-bakes the same mojibake from the still-charset-less fragment.